

Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

III Year CSE (V Semester) | Academic Year 2026-27

UCS3513 - SYSTEM DESIGN LABORATORY

Lab Exercise 3

Improve URL Shortener Performance Using Redis Caching

Submitted by,

Name	Register Number
Tushyent N P	3122 24 5001 189
Vino S R Harrison	3122 24 5001 199
Venkat Prabhu S	3122 24 5001 196
Vignesh Raaj S	3122 24 5001 198

Improve URL Shortener Performance Using Redis Caching

Aim

- To integrate Redis into the URL Shortener developed and evaluate its impact on application performance by comparing URL retrieval with and without caching.
- To implement a cache-aside strategy using Redis as an in-memory cache for URL mappings and analyze cache hit and cache miss scenarios.

Question

Enhance the URL Shortener application by implementing Redis as an in-memory cache for URL mappings with the following requirements:

1. Configure Redis with the Spring Boot application.
2. Cache URL mappings using the short code as the cache key.
3. Implement a cache-aside strategy: Check Redis for the requested short code. If found (Cache Hit), retrieve the long URL from Redis. Otherwise (Cache Miss), retrieve the long URL from the database, store it in Redis, and return the result.
4. Compare the application's performance before and after integrating Redis.
5. Analyze the impact of Redis on response time and database access.

Procedure (Workflow)

- 1) Client sends a POST request to the `/shorten` endpoint with a JSON body containing the `longUrl` field.
- 2) The controller receives and extracts the `longUrl`, forwarding it to the `UrlService`.
- 3) The service checks the database (`repo.findByLongUrl()`) to see if this URL has already been shortened (idempotency).
- 4) If found, the existing short URL is returned immediately.
- 5) If not found, `generateMD5()` computes the MD5 hash of the URL and takes the first 6 hex characters as the short code.
- 6) Collision check: The service verifies uniqueness via `repo.existsByShortCode()`. If the short code already exists, it appends the current timestamp and re-hashes until unique.
- 7) A new `UrlModel` document is created with `longUrl` and `shortCode`, saved to MongoDB, and simultaneously cached in Redis via `redisTemplate.opsForValue().set()`.
- 8) The service returns the generated short URL (`http://localhost:8080/<shortCode>`) as a JSON response.
- 9) For redirection, the client accesses `GET /{shortCode}`. The cache-aside strategy is used: Redis is checked first.
- 10) If found in Redis (Cache Hit), the long URL is returned instantly. Otherwise (Cache Miss), MongoDB is queried, the result is stored in Redis, and returned. A 302 redirect response is sent with the `Location` header set to the original URL.

System Architecture & Workflow

The following diagram illustrates the enhanced architecture of the URL Shortener system with Redis caching, showing the flow from client request through the Spring Boot REST controller, service layer, Redis cache, and MongoDB persistence.

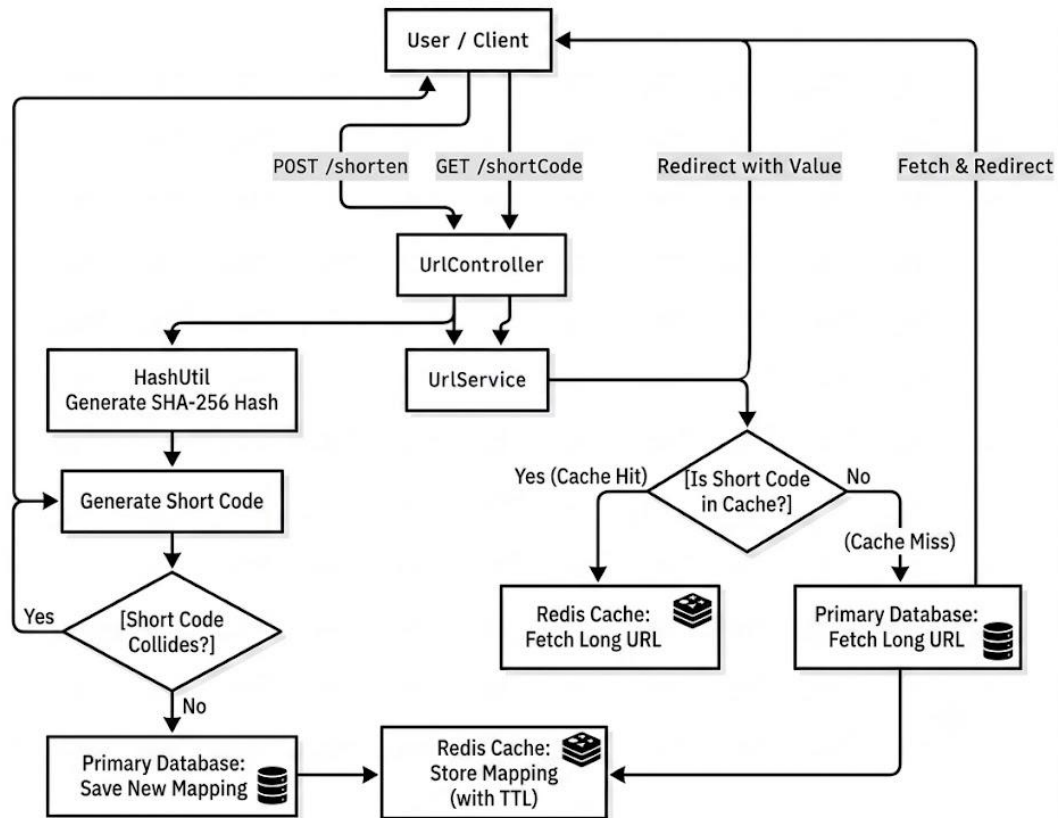


Figure 1: URL Shortener with Spring Boot, MongoDB and Redis Cache

Design of URL Shortener with Redis

1. Redis Configuration & Integration

The `RedisConfig` class defines a `RedisTemplate<String, String>` bean configured with `StringRedisSerializer` for both keys and values. This template is injected into the service layer for all cache operations.

```
@Configuration
public class RedisConfig {
    @Bean
    public RedisTemplate<String, String> redisTemplate(
        RedisConnectionFactory connectionFactory) {
        RedisTemplate<String, String> redisTemplate = new RedisTemplate<>();
        redisTemplate.setConnectionFactory(connectionFactory);
        redisTemplate.setKeySerializer(new StringRedisSerializer());
        redisTemplate.setValueSerializer(new StringRedisSerializer());
        redisTemplate.afterPropertiesSet();

        return redisTemplate;
    }
}
```

Redis is configured in `application.properties` alongside MongoDB:

```
spring.application.name=urlshortner
spring.mongodb.uri=mongodb://localhost:27017/urldb
spring.data.redis.host=localhost
spring.data.redis.port=6379
server.port=8080
```

2. Hashing Technique (MD5)

The system uses MD5 (Message Digest Algorithm 5) via Java's `java.security.MessageDigest` class. The long URL is converted to bytes and hashed, producing a 32-character hexadecimal string. Only the first 6 characters are used as the short code, giving a key space of $16^6 = 16,777,216$ possible short codes.

```
private String generateMD5(String text) {
    try {
        MessageDigest md = MessageDigest.getInstance("MD5");
        byte[] hash = md.digest(text.getBytes(StandardCharsets.UTF_8));
        StringBuilder sb = new StringBuilder();
        for (byte b : hash) {
            sb.append(String.format("%02x", b));
        }
        return sb.substring(0, 6);
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException(e);
    }
}
```

3. Collision Handling Strategy

Collisions occur when two different long URLs produce the same 6-character hash prefix. The system handles this by:

- Checking the database using ``repo.existsByShortCode(shortCode)`` before saving.
- If the short code already exists (collision detected), the current timestamp is appended to the URL and re-hashed (e.g., ``url` + `System.currentTimeMillis()`` → MD5 → new 6-char code).
- The check runs in a while-loop until a unique short code is found.
- This approach guarantees uniqueness without requiring manual intervention.

4. Uniqueness Verification

Two levels of uniqueness are maintained:

- Long URL uniqueness: Before generating a new short code, the system checks if the long URL was already shortened (query by ``longUrl``). If found, the existing short URL is returned — preventing duplicate entries.
- Short code uniqueness: Before saving, the system verifies that the short code does not already exist in the database. If it does, the code is regenerated with a timestamp seed until unique.

5. Cache-Aside Strategy

The system implements the cache-aside pattern for URL retrieval. When a GET request arrives with a short code:

- Redis is queried first using the short code as the key.
- If found (Cache Hit), the long URL is returned instantly from memory — sub-millisecond response.
- If not found (Cache Miss), MongoDB is queried via ``repo.findByShortCode()``. The result is stored in Redis for future requests, then returned.
- The response includes the long URL, cache status (Hit/Miss), and exact execution time in milliseconds.

```
public Map<String, Object> getLongURL(String shortCode) {  
  
    Map<String, Object> response = new HashMap<>();  
  
    long start = System.nanoTime();  
  
    // Check Redis  
    String cachedURL = redisTemplate.opsForValue().get(shortCode);  
  
    if (cachedURL != null) {  
        long end = System.nanoTime();  
        response.put("longUrl", cachedURL);  
        response.put("cache", "Cache Hit");  
    }
```

```

        response.put("responseTime",
            ((end - start) / 1_000_000.0) + " ms");
        return response;
    }

    // Cache Miss
    Optional<URLModel> model = repo.findByShortCode(shortCode);

    if (model.isPresent()) {
        String longUrl = model.get().getLongUrl();
        redisTemplate.opsForValue().set(shortCode, longUrl);
        long end = System.nanoTime();
        response.put("longUrl", longUrl);
        response.put("cache", "Cache Miss");
        response.put("responseTime",
            ((end - start) / 1_000_000.0) + " ms");
        return response;
    }

    return null;
}

```

6. URL Mapping Mechanism with Redis

The mapping between long URLs and short codes follows this flow:

- **Shortening (POST /shorten):**

Long URL → MD5 hash → 6-char prefix → collision check → save to MongoDB → cache in Redis → return short URL

- **Redirect (GET /{shortCode}):**

Short code → check Redis → Cache Hit (return instantly) / Cache Miss (query MongoDB → hydrate Redis → return) → HTTP 302 redirect

7. Data Model (MongoDB Document)

Field	Type	Description
id	String (ObjectId)	Auto-generated MongoDB document identifier
longUrl	String	The original long URL submitted by the user
shortCode	String	The generated unique short code (MD5 hash prefix)

8. Example Document in MongoDB

```

{
  "id": "689df8d9f4c3a5b9c4e7123",
  "longUrl": "https://www.google.com/search?q=system+design",
  "shortCode": "aB12Xy"
}

```

Output Screenshots

The following screenshots demonstrate the URL Shortener with Redis being tested using Postman, with verification in MongoDB Compass and Redis CLI.

1. Creating Short URL (POST /shorten):

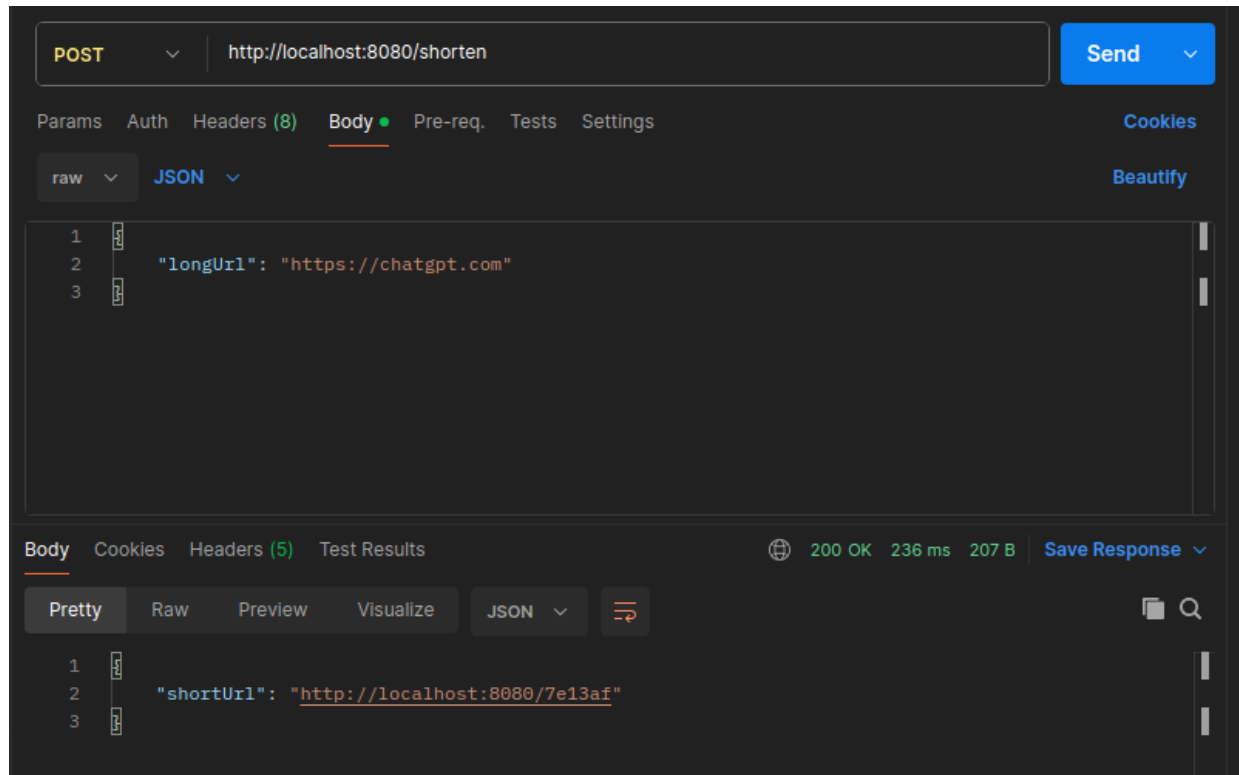


Figure 2: Postman — POST /shorten request and response showing generated short URL

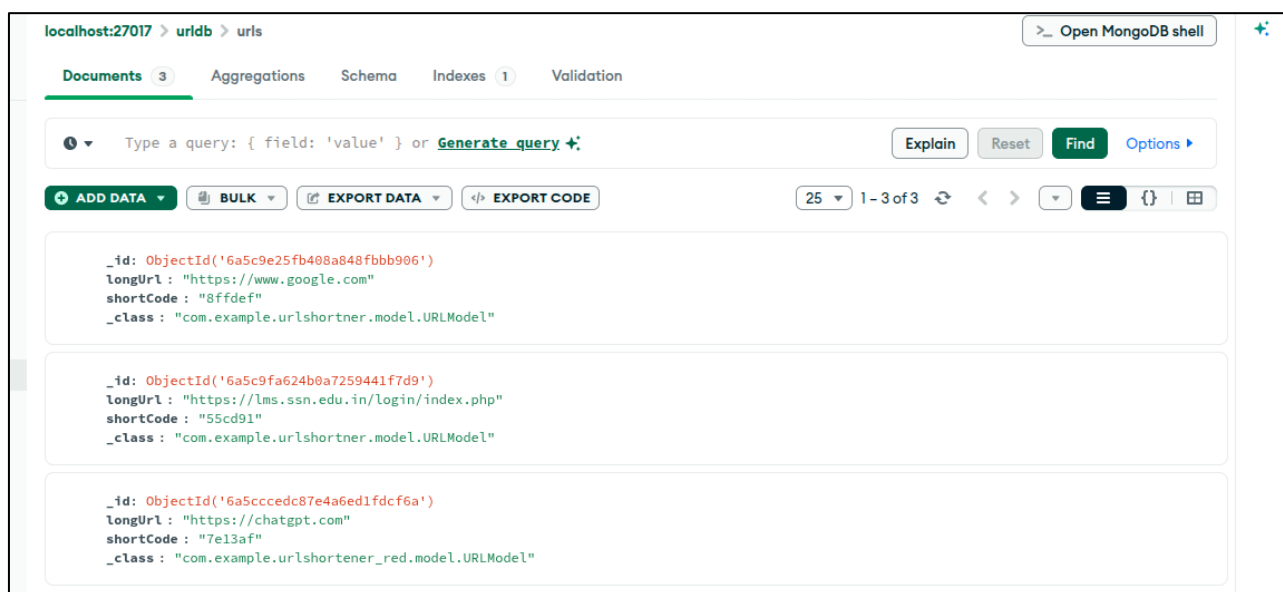


Figure 3: MongoDB Compass — Storing the long URL in the permanent database (urls collection)

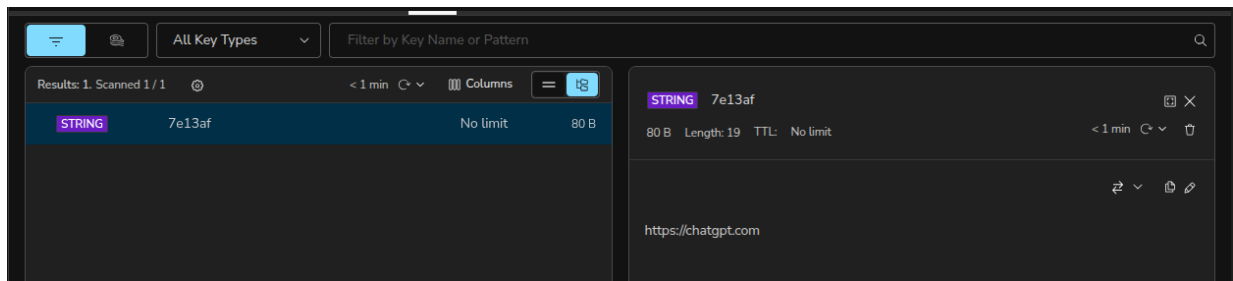


Figure 4: Redis CLI — Long URL cached with short code as the key

2. Cache-Aside Strategy — Cache Miss vs Cache Hit:

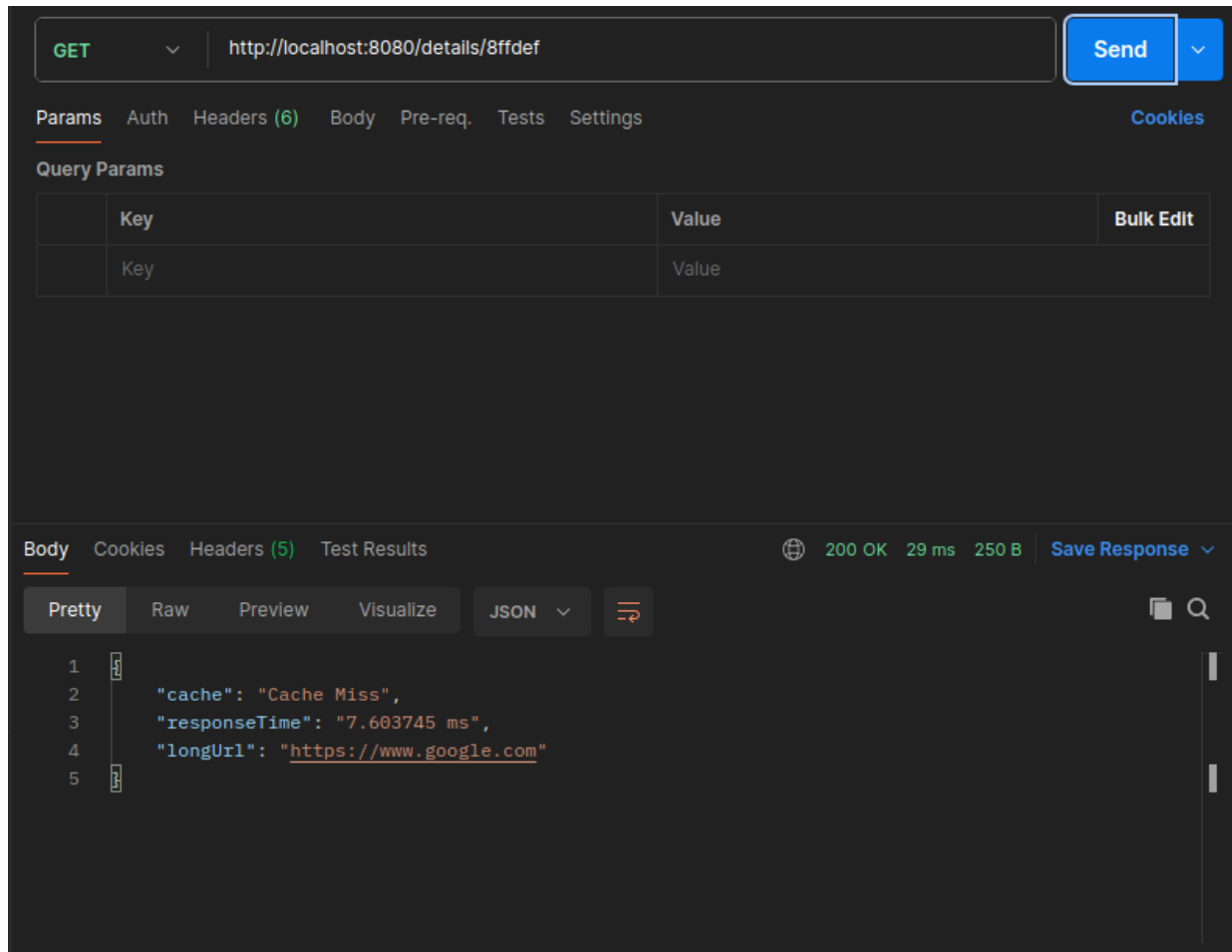


Figure 5: Cache Miss — Initial GET request retrieved from MongoDB in 7.6 ms

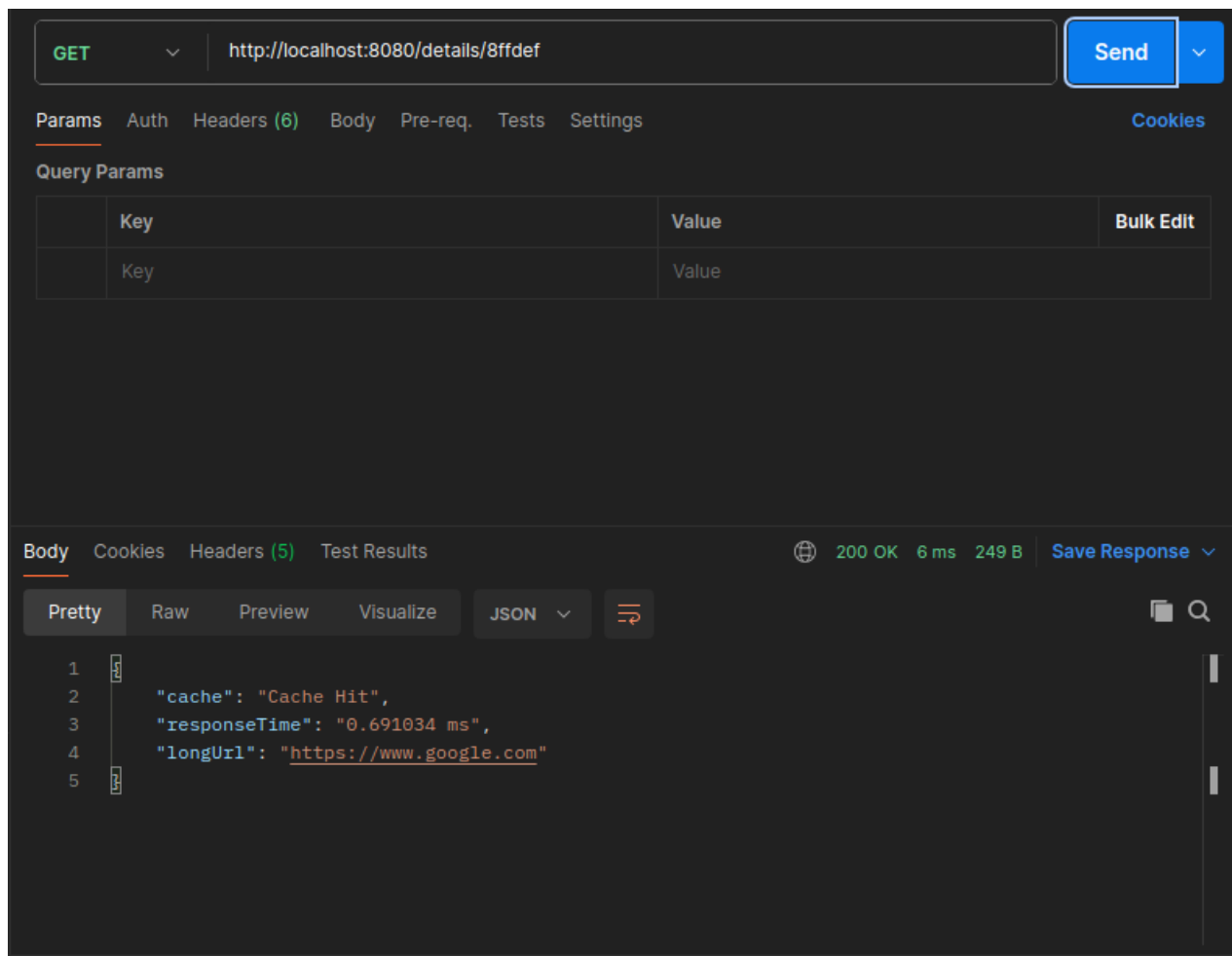


Figure 6: Cache Hit — Subsequent GET request retrieved from Redis in 0.69 ms (11x faster)

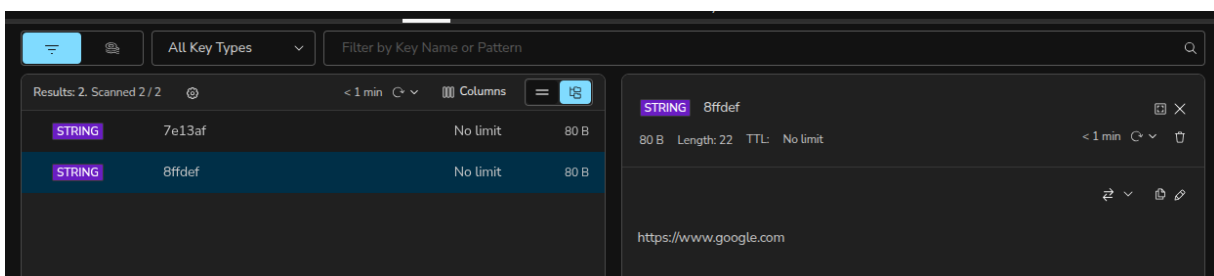


Figure 7: Redis CLI — Cached URL with short code key persisted after cache miss hydration

API Endpoints & Test Results

Table 1: Cache Hit vs Cache Miss Performance Comparison

Scenario	Data Source	Response Time	Speedup
Cache Miss	MongoDB	7.6 ms	1x (baseline)
Cache Hit	Redis (In-Memory)	0.69 ms	~11x faster

Table 2: REST API Endpoints

#	Method	Endpoint	Description	Status Code
1	POST	/shorten	Create shortened URL	200 OK
2	GET	//{shortCode}	Redirect to original URL	302 Found
3	GET	/details/{shortCode}	Get URL details with cache info	200 OK
4	GET	//{shortCode}	Invalid short code	404 Not Found

Table 3: Technology Stack

#	Technology / Tool	Purpose
1	Java 21	Programming language
2	Spring Boot 4.1.0	Framework for REST API development
3	Spring Data MongoDB	Database access and document mapping
4	Spring Data Redis	Redis cache integration and template
5	MongoDB	NoSQL database for URL mappings
6	Redis	In-memory cache for URL mappings
7	Postman	API testing tool

Table 4: Test Case Results

#	Test Case	Input	Expected Result	Status
1	Shorten new URL	{"longUrl":"https://example.com"}	Short URL returned	✓ Pass
2	Shorten same URL again	{"longUrl":"https://example.com"}	Same short URL (idempotent)	✓ Pass
3	Cache Miss → DB retrieval	GET /a1b2c3 (first call)	7.6 ms, "Cache Miss"	✓ Pass
4	Cache Hit → Redis retrieval	GET /a1b2c3 (subsequent)	0.69 ms, "Cache Hit"	✓ Pass

Learning Outcomes

After completing this experiment, the team has gained the following competencies:

- Integrated Redis with a Spring Boot application using Spring Data Redis and RedisTemplate configuration.
- Implemented a cache-aside caching strategy — checking Redis first, falling back to MongoDB on cache miss, and hydrating the cache for subsequent requests.
- Understood the role of caching in improving application performance, achieving ~11x faster response times (7.6 ms → 0.69 ms).
- Compared the performance of database-based retrieval (MongoDB) with Redis-based in-memory retrieval under controlled conditions.
- Analyzed cache hit and cache miss scenarios, including timing measurements and cache hydration behavior.
- Gained hands-on experience with Redis CLI, RedisTemplate operations, and dual-layer persistence (MongoDB + Redis) in a Spring Boot application.